Escuela Politécnica Nacional

Facultad de Ingeniería en Sistemas.

Integrantes:

Betancourt Enrique;

Navarrete Alejandro &

Salazar Ambar.

Mini-informe ejercicio 03

Compiladores y Lenguajes, IDSD413

GR01CD

Ing. Rodrigo Chancusig MSc.

05 de noviembre del 2026

El ejercicio construye un programa que lee un texto desde la entrada estándar y muestra cuántas líneas y caracteres detecta. La especificación se escribe en tres.l; Lex genera el código C, que luego se compila con GCC. Las capturas documentan el código, la compilación y una ejecución con un archivo de prueba.

1. Objetivo y funcionamiento

El objetivo es reconocer dos tipos de entrada: el salto de línea y cualquier otro carácter. En la sección de reglas, [\n] incrementa numlin y numcar; el punto (.) incrementa numcar para cada carácter que no sea un salto de línea. Las variables globales comienzan en cero. La función yylex() procesa la entrada y, al terminar, main imprime ambos acumulados.

```
%{
        #include<stdlib.h>
        #include<stdio.h>
        #include<string.h>
        int numlin=0, numcar=0;
%}
%%
[\n]    numlin++;
.       numcar++;
%%
int main()
{
        yylex();
        printf("Número de líneas %d\n", numlin);
        printf("Número de caracteres %d\n",numcar);
        return 0;
}
```

Figura 1. Especificación Lex y función principal del contador.

Así, el programa cuenta cada salto de línea como una línea y también como un carácter. El patrón . cubre letras, espacios, números y signos distintos de salto de línea. La definición de “línea” depende de que exista el carácter de nueva línea: una última línea sin salto final puede no sumarse a numlin.

2. Preparación y compilación

Primero se crea la carpeta de trabajo y se guarda la especificación en tres.l. El comando cat tres.l permite revisar el archivo y comprobar que contiene las reglas y la función principal. En la terminal se ven intentos iniciales con órdenes incompletas; la secuencia que produce los archivos ejecutables es lex tres.l y después gcc lex.yy.c -lfl.

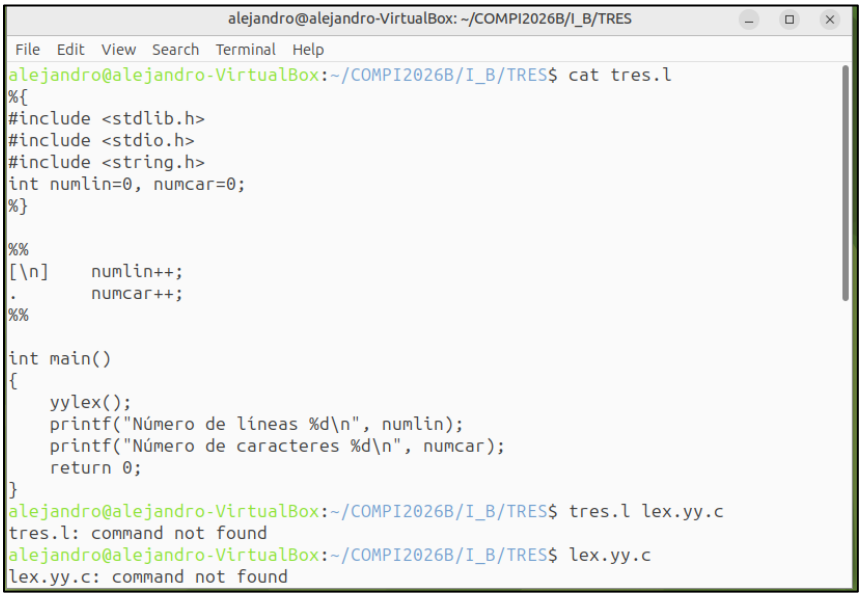


Figura 2. Revisión del archivo y primeros intentos en la terminal.

Lex transforma tres.l y genera lex.yy.c. GCC compila ese código generado y, con -lfl, enlaza la biblioteca de Flex necesaria para las funciones del analizador. Si la compilación finaliza sin errores, se obtiene a.out. El comando ls muestra el ejecutable junto con tres.l y lex.yy.c.

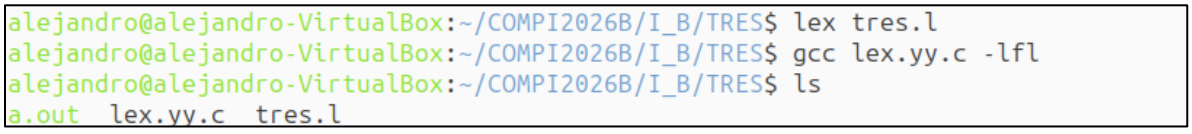


Figura 3. Generación del código C, compilación y archivos resultantes.

3. Archivo de prueba

Para probar el contador se crea prueba.txt con cuatro renglones de texto mediante una entrada multilínea en la terminal. Luego se usa cat prueba.txt para confirmar el contenido. La captura permite observar palabras y espacios que serán procesados por las reglas del analizador.

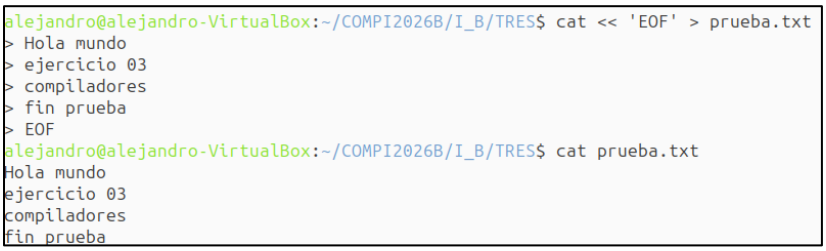


Figura 4. Creación y revisión del contenido de prueba.txt.

A continuación, mv prueba.txt entrada.txt cambia el nombre del archivo para usarlo como entrada del programa. La orden ls confirma que entrada.txt está en la carpeta de trabajo junto con a.out y los archivos generados por Lex. Este paso prepara una entrada identificable para la redirección de la siguiente ejecución.

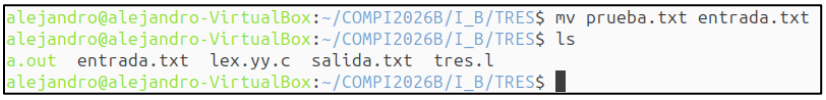


Figura 5. Renombrado del archivo y verificación de los archivos disponibles.

4. Ejecución y análisis del resultado

La orden ./a.out < entrada.txt > salida.txt ejecuta el contador, toma los datos desde entrada.txt y guarda lo que imprime en salida.txt. Después, cat salida.txt muestra en pantalla el resumen calculado. En la evidencia se reportan 4 líneas y 44 caracteres.

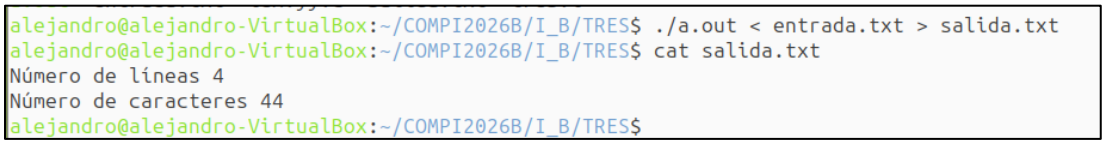


Figura 6. Ejecución con redirección y salida observada.

5. Comparación con código generado por IA.

Considerando lo mencionado en clase, es decir, la tarea de buscar un programa lex para contabilizar líneas y caracteres en base a una inteligencia artificial se obtuvo las siguientes diferencias claves:

El código proporcionado en las diapositivas de clase procesa el texto de forma iterativa carácter por carácter usando el punto (.) y omite contabilizar el salto de línea (\n) dentro del total de caracteres. En contraste, la versión modificada optimiza la ejecución al leer bloques enteros de texto de golpe usando [^\n]+ y sumando su tamaño con yyleng, sí incluye los saltos de línea en el conteo total, y añade tanto yywrap() como manejo de argumentos (argc/argv) en el main para que sea posible compilar sin librerías extra.

```
alejandro@alejandro-VirtualBox:~/Compiladores-y-Lenguajes-2026B/I_B/TRES_IA$ cat tres_ia.l
%{
#include <stdio.h>

int num_lineas = 0;
int num_caracteres = 0;
%}

%%

\n          { num_lineas++; num_caracteres++; }
[^\n]+      { num_caracteres += yyleng; }

%%

int main(int argc, char **argv) {
    // Permite leer un archivo si se pasa por consola, o usar la entrada estándar
    if (argc > 1) {
        FILE *archivo = fopen(argv[1], "r");
        if (archivo) {
            yyin = archivo;
        } else {
            printf("No se pudo abrir el archivo.\n");
            return 1;
        }
    }

    yylex();

    printf("Número de lineas: %d\n", num_lineas);
    printf("Número de caracteres: %d\n", num_caracteres);

    return 0;
}

int yywrap() {
    return 1;
}
alejandro@alejandro-VirtualBox:~/Compiladores-y-Lenguajes-2026B/I_B/TRES_IA$
```

Finalmente se indica los resultados obtenidos a partir de este archivo, como se puede observar, se obtiene la misma cantidad de líneas con una misma entrada tanto para el código de las diapositivas como para el código generado por IA, la diferencia fundamental que existe es el número de caracteres, donde, el código generado por IA considera también los saltos de línea de ese modo se incrementa en 4 caracteres a una misma entrada.

```
alejandro@alejandro-VirtualBox:~/Compiladores-y-Lenguajes-2026B/I_B/TRES_IA$ nano entrada.txt
alejandro@alejandro-VirtualBox:~/Compiladores-y-Lenguajes-2026B/I_B/TRES_IA$ ./a.out < entrada.txt > salida.txt
alejandro@alejandro-VirtualBox:~/Compiladores-y-Lenguajes-2026B/I_B/TRES_IA$ cat salida.txt
Número de lineas: 4
Número de caracteres: 48
alejandro@alejandro-VirtualBox:~/Compiladores-y-Lenguajes-2026B/I_B/TRES_IA$ cat entrada.txt
Hola mundo
ejercicio 03
compiladores
fin prueba
alejandro@alejandro-VirtualBox:~/Compiladores-y-Lenguajes-2026B/I_B/TRES_IA$
```

6. Conclusión

La práctica consolida el flujo para la construcción de un analizador léxico: la definición de patrones, la generación de código en C, su compilación y la validación mediante redirección de archivos. Si bien la ejecución confirma correctamente las cuatro líneas del texto, la variación en el conteo final de caracteres nos muestra una lección esencial en el diseño de compiladores: el tratamiento de los caracteres invisibles del sistema operativo. El valor de 44 arrojado en la captura refleja estrictamente las letras y espacios en blanco visibles de los cuatro renglones, mientras que otro análisis del archivo revela 48 caracteres al incluir los saltos de línea (\n). Esto demuestra que, al construir un autómata, es indispensable definir desde las reglas léxicas si los caracteres de formato y control deben ser contabilizados o ignorados.